

Elmoghazi Clinic

Complete Application Specification المواصفات الكاملة للتطبيق

Next.js 16

TypeScript

Prisma

PostgreSQL

Tailwind CSS 4

Disaster Recovery & Rebuild Guide

عيادة المغازي للجلدية والتجميل

Generated: 2026-06-18



ELMOGHAZI CLINIC — Complete Application

Specification

عيادة المغازي للجلدية والتجميل — المواصفات الكاملة للتطبيق

⚠️ مهم: هذا الملف هو البصمة الكاملة للتطبيق. لو ضاع التطبيق أو عايز تنسخه على منصة ثانية، ابعت الملف ده وأى مطور يقدر يعيد بناء نفس التطبيق بالضبط بنفس كل المميزات.

📋 Table of Contents | فهرس المحتويات

- نظرة عامة | Overview
- التقنيات المستخدمة | Tech Stack
- كل المميزات | All Features
- قاعدة البيانات | Database Schema
- واجهات البرمجة | API Endpoints
- الدوال المهمة | Key Code Functions
- هيكل الملفات | File Structure
- المتغيرات البيئية | Environment Variables
- ملفات الإعدادات | Configuration Files
- ملاحظات التصميم | UI/UX Design Notes
- الإصلاحات المطبقة | Bug Fixes Applied
- دليل النشر | Deployment Guide
- دليل الاستعادة من الكوارث | Recovery Guide
- تسجيل الدخول والصلاحيات | Login & Auth
- المنطق التشغيلي | Business Logic

1. Overview | نظرة عامة

اسم التطبيق: Elmoghazi Clinic — عيادة المغازي للجلدية والتجميل

الوصف: نظام إدارة عيادة جلدية وتجميل متكامل، باللغة العربية مع دعم RTL كامل. مصمم لعيادة المغازي في مصر، مع دعم كامل لتوقيت القاهرة.

المنصات المستخدمة حالياً:

Frontend & Backend: Vercel (Next.js 16)

Database: Neon PostgreSQL (500 مجاني MB)

Git Repository: GitHub — [sayedhassan589-art/elmoghazi-clinic](https://github.com/sayedhassan589-art/elmoghazi-clinic)

2. Tech Stack | التقنيات المستخدمة

Core Technologies:

```
{
  "framework": "Next.js 16.1.1",
  "language": "TypeScript 5",
  "runtime": "React 19",
  "styling": "Tailwind CSS 4",
  "ui_library": "shadcn/ui (Radix UI based)",
  "database_orm": "Prisma 6.19.3",
  "database": "PostgreSQL (Neon)",
  "state_management": "Zustand 5 (with persist middleware)",
  "animations": "Framer Motion 12",
  "charts": "Recharts 2.15",
  "icons": "Lucide React 0.525",
  "toasts": "Sonner 2.0",
  "fonts": "Cairo (Arabic) + Playfair Display (Latin)",
  "forms": "React Hook Form 7 + Zod 4",
  "auth": "Custom (no NextAuth – simple password-based)"
}
```

Full Dependencies (package.json):

```
{
  "dependencies": {
    "@dnd-kit/core": "^6.3.1",
    "@dnd-kit/sortable": "^10.0.0",
    "@dnd-kit/utilities": "^3.2.2",
    "@hookform/resolvers": "^5.1.1",
    "@mdxeditor/editor": "^3.39.1",
    "@prisma/client": "^6.19.3",
    "@radix-ui/react-accordion": "^1.2.11",
    "@radix-ui/react-alert-dialog": "^1.1.14",
    "@radix-ui/react-aspect-ratio": "^1.1.7",
    "@radix-ui/react-avatar": "^1.1.10",
    "@radix-ui/react-checkbox": "^1.3.2",
    "@radix-ui/react-collapsible": "^1.1.11",
    "@radix-ui/react-context-menu": "^2.2.15",
    "@radix-ui/react-dialog": "^1.1.14",
    "@radix-ui/react-dropdown-menu": "^2.1.15",
    "@radix-ui/react-hover-card": "^1.1.14",
    "@radix-ui/react-label": "^2.1.7",
    "@radix-ui/react-menubar": "^1.1.15",
    "@radix-ui/react-navigation-menu": "^1.2.13",
    "@radix-ui/react-popover": "^1.1.14",
    "@radix-ui/react-progress": "^1.1.7",
    "@radix-ui/react-radio-group": "^1.3.7",
    "@radix-ui/react-scroll-area": "^1.2.9",
    "@radix-ui/react-select": "^2.2.5",
    "@radix-ui/react-separator": "^1.1.7",
    "@radix-ui/react-slider": "^1.3.5",
    "@radix-ui/react-slot": "^1.2.3",
    "@radix-ui/react-switch": "^1.2.5",
    "@radix-ui/react-tabs": "^1.1.12",
    "@radix-ui/react-toast": "^1.2.14",
    "@radix-ui/react-toggle": "^1.1.9",
    "@radix-ui/react-toggle-group": "^1.1.10",
    "@radix-ui/react-tooltip": "^1.2.7",
    "@reactuses/core": "^6.0.5",
    "@tanstack/react-query": "^5.82.0",
    "@tanstack/react-table": "^8.21.3",
    "class-variance-authority": "^0.7.1",
    "clsx": "^2.1.1",
    "cmdk": "^1.1.1",
    "date-fns": "^4.1.0",
    "embla-carousel-react": "^8.6.0",
    "framer-motion": "^12.23.2",
    "input-otp": "^1.4.2",
    "lucide-react": "^0.525.0",
    "next": "^16.1.1",
    "next-auth": "^4.24.11",
    "next-intl": "^4.3.4",
    "next-themes": "^0.4.6",
    "prisma": "^6.19.3",
    "react": "^19.0.0",
    "react-day-picker": "^9.8.0",
    "react-dom": "^19.0.0",
    "react-hook-form": "^7.60.0",
    "react-markdown": "^10.1.0",
    "react-resizable-panels": "^3.0.3",
    "react-syntax-highlighter": "^15.6.1",
    "recharts": "^2.15.4",
  }
}
```

```

"sharp": "^0.34.3",
"sonner": "^2.0.6",
"tailwind-merge": "^3.3.1",
"tailwindcss-animate": "^1.0.7",
"uuid": "^11.1.0",
"vaul": "^1.1.2",
"xlsx": "^0.18.5",
"z-ai-web-dev-sdk": "^0.0.17",
"zod": "^4.0.2",
"zustand": "^5.0.6"
},
"devDependencies": {
"@tailwindcss/postcss": "^4",
"@types/react": "^19",
"@types/react-dom": "^19",
"bun-types": "^1.3.4",
"eslint": "^9",
"eslint-config-next": "^16.1.1",
"tailwindcss": "^4",
"tw-animate-css": "^1.3.5",
"typescript": "^5"
}
}

```

Scripts:

```

{
  "dev": "next dev -p 3000",
  "postinstall": "prisma generate",
  "build": "prisma generate && next build",
  "start": "NODE_ENV=production bun .next/standalone/server.js",
  "lint": "eslint .",
  "db:push": "prisma db push",
  "db:generate": "prisma generate",
  "db:migrate": "prisma migrate dev",
  "db:reset": "prisma migrate reset"
}

```

3. All Features | كل المميزات

3.1 Dashboard — لوحة الرئيسية

Quick Stats Cards: إجمالي المرضى، زيارات اليوم، إيراد اليوم، مواعيد اليوم، جلسات اليوم، سجلات الليزر

End-of-Day Summary: ملخص نهاية اليوم مع

إيراد الكشف (todayCheckupRev)

إيراد الإعادة (todayRevisitRev)

إيراد الجلسات/الليزر/المتابعة (todaySessionRev)

إجمالي الإيراد + المصروف + الصافي

زرار طباعة الملخص (window.print())

Quick Actions: مريض جديد، سجل ليزر، معاملة، موعد، مساعد ذكي، بحث ذكي

AI Assistant Chat: z-ai-web-dev-sdk مساعد باستخدام AI دردشة مع

3.2 Patients — إدارة المرضى

Smart Patient Form: نموذج تسجيل مريض جديد ذكي

الاسم، الهاتف (مع normalize للأرقام العربية)، الهاتف 2، العنوان، العمر، التشخيص، ملاحظات

اختيار نوع الزيارة: كشف / إعادة / جلسة / كشف+جلسة / إعادة+جلسة

سعر الكشف/الإعادة (يدوي)

سعر الخدمة للجلسات (يدوي)

اختيار متعدد للخدمات مصنفة حسب الفئة

تاريخ مخصص (اختياري — افتراضي اليوم بتوقيت القاهرة)

Smart Search: بحث ذكي عبر كل حقول المريض

الاسم، العنوان، التشخيص، الهاتف، الملاحظات

يشمل تشخيصات وملاحظات الزيارات السابقة

يشمل ملاحظات الجلسات

بحث fuzzy matching مع تطبيع النصوص العربية

Filter Chips: فلترة حسب الحقل (الكل/الاسم/العنوان/التشخيص/الهاتف/الملاحظات)

Patient Cards: بطاقات مرضى عصرية تعرض

الاسم + رقم الملف

الهاتف (مع زرار WhatsApp)

العنوان + التشخيص

شارات النجمة والتحسين واللون

Patient Profile: ملف مريض تفصيلي بتبويبات

نظرة عامة (Overview): معلومات أساسية + آخر تشخيص + Timeline (زيارات/جلسات/ملاحظات)

الزيارات (Visits): إضافة/تعديل/حذف زيارات (كشف/إعادة/جلسة) مع سعر تلقائي (كشف=200، إعادة=80)

الجلسات (Sessions): إضافة/تعديل/حذف جلسات مع خدمة وسعر

الليزر (Laser): عرض سجلات الليزر المرتبطة

التذكيرات (Reminders): تذكيرات المريض

الملاحظات (Notes): ملاحظات سريعة مع قسم خاص بالمريض

الصور (Photos): صور (base64) before/after/general

المتابعة (Follow-up): سجلات المتابعة المرتبطة

Patient Actions: (10-1 slider with history) حذف (مع حذف كل البيانات المرتبطة)، تمييز بنجمة، تحديث التحسن

Color Tags: وسم المرضى بألوان مخصصة

Export: JSON/CSV تصدير المرضى إلى

Import: JSON استيراد مرضى من

3.3 Laser — إدارة الليزر

Laser Records: سجلات الليزر لكل مريض تحتوي على

منطقة الجسم (+40 منطقة معربة)

نوع البشرة (I-VI + أنواع أخرى)

لون الشعر (16 لون)

كثافة الشعر

عدد الجلسات الكلية + السعر

اسم الجهاز، الطاقة، النبض

الحالة (نشط/مكتمل/متوقف)

Laser Sessions: جلسات الليزر الفرعية

رقم الجلسة، الطاقة، النبض، مستوى الألم

التفاعل، الملاحظات، السعر، حالة الدفع

دفع تلقائي للجلسة → ينشئ معاملة مالية بفئة "ليزر"

Laser Packages: باقات الليزر (اسم، عدد جلسات، سعر، منطقة)

Laser Settings: إعدادات افتراضية لكل جهاز ومنطقة

Laser Notes: ملاحظات على سجل الليزر

Body Areas (40+): قائمة شاملة لمناطق الجسم بالعربية مع ألوان مميزة

الوجه، الجبين، الخدود، الأنف، الأذنين، السوالم

الرقبة (أمامية/خلفية/كاملة)، الكتفين

الذراعين (علوية/سفلية/كاملة)، اليدين، الأصابع، الإبط

الصدر، بين الثديين، البطن (علوية/سفلية/كاملة)، خط السرة

الظهر (علوي/سفلي/كامل)

البيكيني (عادي/كامل)، الأرداف

الفخذين (أمامي/خلفي/كاملة)، الساقين، الساق الأمامية، الرجلين كاملة

القدمين، أصابع القدم

جسم كامل، نصف الجسم العلوي/السفلي

💰 3.4 Finance — الإدارة المالية

Auto-Recording: تسجيل تلقائي للمعاملات المالية عند

إضافة زيارة كشف/إعادة من ملف المريض

إضافة جلسة من ملف المريض

دفع جلسة ليزر

دفع زيارة متابعة

تسجيل مريض جديد مع زيارة/جلسة

Categories: فئات المعاملات

كشف (consultation)

إعادة (revisit)

جلسات (sessions)

ليزر (laser)

متابعة (follow-up)

(شخصي — مستثناة من إيراد العيادة) **personal**

clinic (default)

Transaction Types: income / expense

Daily Summary: ملخص يومي مع

إيراد الكشف + الإعادة + الجلسات

مصرفات اليوم

صافي اليوم

عرض تفصيلي لكل يوم قابل للتوسيع

إضافة معاملة يدوية (نوع + فئة + مبلغ + وصف + تاريخ) **Manual Transactions:**

تقارير يومية/أسبوعية/شهرية **Reports:**

رسم بياني للإيراد والمصروف على مدار الأسبوع المصري (السبت-الجمعة) **Charts:**

قسم المزيد (16 تبويب فرعي) — 3.5 More Section

المتابعات (Follow-up): نظام متابعة كامل:

سجلات المتابعة (الحالة، التصنيف، الخطورة، التكرار)

باقات المتابعة (اشتراك شهري/ربع سنوي/سنوي/بالجلسة)

زيارات المتابعة (نتائج سريرية، علامات حيوية، تشخيص، أدوية)

تذكير تلقائي قبل الموعد

الخدمات (Services): إدارة الخدمات (اسم، فئة، سعر، مدة)

الجلسات (Sessions): كل الجلسات مع فترة وروابط لملفات المرضى

الزيارات (Visits): كل الزيارات مع روابط

الأطباء (Doctors): الأطباء الشركاء مع نسبهم (كشف/إعادة/إليزر/جلسات + مبلغ ثابت)

المخزون (Inventory): إدارة المخزون مع تنبيهات الكمية الدنيا + توريد/صرف

الحجز (Bookings): نظام المواعيد والحجوزات

الأدوية (Medications): قاعدة بيانات الأدوية (كريم/مرهم/أقراص/حقن)

التذكيرات (Reminders): تذكيرات عامة (عاجل/مهم/متابعة/عام)

القوالب (Templates): قوالب خطط العلاج (جلسات + سعر تقديري)

الانتظار (Waiting Queue): قائمة الانتظار مع الأولويات

التقارير (Reports): تقارير يومية/أسبوعية/شهرية

النسخ الاحتياطي (Backup):

نسخ احتياطي يدوي وتلقائي

استعادة من ملف JSON

تصدير كامل لكل البيانات

الملاحظات (Notes): ملاحظات سريعة لكل قسم

شخصي (Personal): معاملات وملاحظات وتذكيرات شخصية (مستثناة من مالية العيادة)

الإعدادات (Settings): تغيير ألوان التطبيق (10 ثيمات)

البحث الذكي — 3.6 Smart Search

يبحث عبر: المرضى، الخدمات، الملاحظات، التذكيرات، الملاحظات الشخصية

تطبيع النصوص العربية (أ → ا، ه → ه، ي → ي، إزالة التشكيل)

تطابق مرن للأحرف: Fuzzy matching

للأداء Debounced (250ms)

نافذة منبثقة مع نتائج فورية

📱 3.7 WhatsApp Integration

روابط WhatsApp مباشرة من أرقام المرضى

waPhone() function: تحويل الرقم لصيغة دولية (20+ لمصر)

normalizePhone(): تحويل الأرقام العربية/الفارسية إلى لاتينية :

يعمل مع: ٠١٢٣٤٥٦٧٨٩ (عربي)، ٠١٢٣٤٥٦٧٨٩ (فارسي)، + (عربي)

🎨 3.8 UI/UX Features

دعم كامل للعربية من اليمين لليسار **RTL Support:**

وضع ليلي كامل **Dark Mode:**

10 Themes: زمردى، أزرق ملكي، ذهبي وردي، بنفسجي، غروب، تركوازي، ياقوتي، غابات، ليلي، شمباني

Animations: Framer Motion (hover, tap, page transitions) في كل مكان

Responsive: يعمل على الموبايل والتابلت والكمبيوتر

Toasts: Sonner إشعارات (نجاح/خطأ/تحذير)

Luxury Design: مع gradients، shadows، blur effects تصميم فاخر

Fonts: Cairo (عربي) + Playfair Display (لاتيني) + Noto Sans SC fallback

🔒 3.9 Auth & Roles

كلمة مرور واحدة (2137) لكلا الدورين **Login:**

Roles:

طبيب (doctor): دخول كامل لكل الأقسام

سكرتيرة (secretary): المرضى والليزر فقط

Persisted: Zustand persist middleware (localStorage)

Auto-redirect: السكرتيرة تدخل تلقائياً لقسم المرضى

💾 3.10 Backup & Restore

واحد JSON تصدير كل البيانات إلى ملف **Full Export:**

Full Import: مع JSON استعادة من ملف

حذف البيانات الموجودة بترتيب عكسي التبعيات

إعادة الإنشاء بترتيب التبعيات

تخطي السجلات المكررة

إزالة الحقول الافتراضية (virtual fields)

دقيقة (قابل للتخصيص) X نسخ احتياطي تلقائي كل **Auto Backup:**

🤖 3.11 AI Assistant

دردشة مع AI باستخدام z-ai-web-dev-sdk

إجراءات سريعة مقترحة (مواعيد، تقارير، مرضى، إلخ)

يدعم السياق متعدد الأدوار

4. Database Schema | قاعدة البيانات

Provider: PostgreSQL (Neon)

Schema file: `prisma/schema.prisma`

Generator & Datasource:

```
generator client {
  provider = "prisma-client-js"
}

datasource db {
  provider = "postgresql"
  url      = env("DATABASE_URL")
}
```

All Models (30 models):

4.1 User

```
model User {
  id          String   @id @default(cuid())
  name        String
  email       String   @unique
  password    String
  role        String   @default("secretary") // doctor, secretary, admin
  active      Boolean  @default(true)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  visits      Visit[]
  sessions    Session[]
  notes       Note[]
  auditLogs   AuditLog[]
  prescriptions Prescription[]
}
```

4.2 Patient

```
model Patient {
  id          String    @id @default(cuid())
  fileNumber  String    @unique
  name        String
  phone       String?
  phone2      String?
  age         Int?
  gender       String?  // male, female
  bloodType   String?
  address     String?
  notes       String?
  allergies   String?
  medicalHistory String?
  starred     Boolean   @default(false)
  improved    Boolean   @default(false)
  colorTag    String?  // custom color tag for patient status
  improvementScore Int?  // مؤشر التحسن من 1 إلى 10
  improvementHistory String? // سجل تغييرات التحسن (JSON string)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  visits      Visit[]
  sessions    Session[]
  alerts       Alert[]
  patientNotes Note[]
  laserRecords LaserRecord[]
  appointments Appointment[]
  photos       PatientPhoto[]
  treatmentPlans TreatmentPlan[]
  reminders    Reminder[]
  waitingQueue WaitingQueue[]
  prescriptions Prescription[]
  followUpRecords FollowUpRecord[]
}
```

4.3 Service

```
model Service {
  id          String    @id @default(cuid())
  name        String
  category     String?  // consultation, laser, treatment, cosmetic
  price        Float    @default(0)
  duration     Int?     // minutes
  active       Boolean   @default(true)
  createdAt    DateTime @default(now())
  updatedAt    DateTime @updatedAt

  sessions    Session[]
}
```

4.4 Visit

```
model Visit {
  id          String   @id @default(cuid())
  patientId   String
  doctorId    String?
  type        String   @default("consultation") // consultation, followup, emergency
  diagnosis   String?
  notes       String?
  date        DateTime @default(now())
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  patient     Patient @relation(fields: [patientId], references: [id], onDelete: Cascade)
  doctor      User?   @relation(fields: [doctorId], references: [id])
}
```

4.5 Session

```
model Session {
  id          String   @id @default(cuid())
  patientId   String
  serviceId   String?
  doctorId    String?
  status      String   @default("scheduled") // scheduled, completed, cancelled, no_show
  notes       String?
  date        DateTime @default(now())
  price       Float    @default(0)
  paid        Boolean  @default(false)
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  patient     Patient @relation(fields: [patientId], references: [id], onDelete: Cascade)
  service     Service? @relation(fields: [serviceId], references: [id])
  doctor      User?   @relation(fields: [doctorId], references: [id])
  treatmentPlan TreatmentPlanSession[]
}
```

4.6 Note

```
model Note {
  id          String   @id @default(cuid())
  patientId   String?
  userId      String?
  content     String
  important    Boolean  @default(false)
  section     String?  // dashboard, patients, visits, sessions, laser, finance, etc.
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  patient     Patient? @relation(fields: [patientId], references: [id], onDelete: Cascade)
  user        User?    @relation(fields: [userId], references: [id])
}
```

4.7 Alert

```
model Alert {
  id          String    @id @default(cuid())
  patientId   String
  type        String    @default("warning") // warning, danger, info
  message     String
  active      Boolean    @default(true)
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  patient     Patient    @relation(fields: [patientId], references: [id], onDelete: Cascade)
}
```

4.8 Reminder

```
model Reminder {
  id          String    @id @default(cuid())
  patientId   String?
  title       String
  description  String?
  date        DateTime
  type        String    @default("follow_up") // follow_up, appointment, medication, custom
  status      String    @default("pending") // pending, sent, done, dismissed
  sentVia     String?   // whatsapp, sms, in_app
  createdAt   DateTime   @default(now())
  updatedAt   DateTime   @updatedAt

  patient     Patient?   @relation(fields: [patientId], references: [id], onDelete: Cascade)
}
```

4.9-4.13 Laser Models

```
model LaserRecord {
  id                String    @id @default(cuid())
  patientId         String
  bodyArea          String
  skinType          String?   // I, II, III, IV, V, VI
  hairColor         String?
  hairDensity       String?   // low, medium, high
  totalSessions     Int       @default(0)
  price             Float     @default(0) // سعر الجلسة
  totalPrice        Float     @default(0) // إجمالي سعر الباقية
  paid              Boolean   @default(false) // حالة الدفع
  machineName       String?   // اسم الجهاز
  energy            Float?    // الطاقة
  pulse             String?   // النبض
  status            String    @default("active") // active, completed, paused
  notes             String?
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt

  patient           Patient    @relation(fields: [patientId], references: [id], onDelete: Cascade)
  laserSessions     LaserSession[]
  laserNotes        LaserNote[]
}

model LaserSession {
  id                String    @id @default(cuid())
  laserRecordId     String
  sessionNumber     Int
  energy            Float?    // joules
  pulse             String?
  painLevel         Int?      // 1-10
  reaction          String?
  notes             String?
  price             Float     @default(0) // سعر الجلسة
  paid              Boolean   @default(false) // هل تم الدفع
  date              DateTime  @default(now())
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt

  laserRecord       LaserRecord @relation(fields: [laserRecordId], references: [id], onDelete: Cascade)
}

model LaserPackage {
  id                String    @id @default(cuid())
  name              String
  sessionsCount     Int
  price             Float
  bodyArea          String?
  active            Boolean   @default(true)
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
}

model LaserNote {
  id                String    @id @default(cuid())
  laserRecordId     String
  content           String
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt
}
```

```

    laserRecord    LaserRecord @relation(fields: [laserRecordId], references: [id], onDelete: Cascade)
  }

  model LaserSetting {
    id              String    @id @default(cuid())
    machineName     String
    bodyArea        String
    skinType        String?
    defaultEnergy   Float?
    defaultPulse    String?
    notes           String?
    createdAt       DateTime @default(now())
    updatedAt       DateTime @updatedAt
  }

```

4.14 Transaction

```

model Transaction {
  id          String    @id @default(cuid())
  type        String    @default("income") // income, expense
  category    String    @default("clinic") // clinic, personal, متابعة, ليزر, جلسات, إعادة, كشف
  amount      Float
  description String?
  date        DateTime @default(now())
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}

```

4.15-4.20 Other Models

```
model Appointment {
  id          String   @id @default(cuid())
  patientId   String?
  date        DateTime
  duration     Int      @default(30) // minutes
  type        String   @default("consultation")
  status      String   @default("scheduled") // scheduled, confirmed, completed, cancelled, no_show
  notes       String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  patient     Patient? @relation(fields: [patientId], references: [id], onDelete: Cascade)
}

model WaitingQueue {
  id          String   @id @default(cuid())
  patientId   String?
  patientName String?
  priority     Int      @default(0)
  status      String   @default("waiting") // waiting, in_progress, done, left
  notes       String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  patient     Patient? @relation(fields: [patientId], references: [id], onDelete: Cascade)
}

model InventoryItem {
  id          String   @id @default(cuid())
  name        String
  category    String?
  quantity     Int      @default(0)
  minQuantity  Int      @default(5)
  unitPrice    Float    @default(0)
  notes       String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  transactions InventoryTransaction[]
}

model InventoryTransaction {
  id          String   @id @default(cuid())
  itemId      String
  type        String   // in, out
  quantity     Int
  notes       String?
  date        DateTime @default(now())
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt

  item        InventoryItem @relation(fields: [itemId], references: [id], onDelete: Cascade)
}

model PatientPhoto {
  id          String   @id @default(cuid())
  patientId   String
  type        String   @default("general") // before, after, general
  description String?
```



```

    imageData    String    // base64
    createdAt    DateTime  @default(now())
    updatedAt    DateTime  @updatedAt

    patient      Patient   @relation(fields: [patientId], references: [id], onDelete: Cascade)
}

model Backup {
  id            String    @id @default(cuid())
  type          String    @default("manual") // manual, auto
  size          Int?
  status        String    @default("completed") // completed, failed
  data          String    // JSON string of backup
  createdAt     DateTime  @default(now())
}

model AuditLog {
  id            String    @id @default(cuid())
  userId        String?
  action        String
  entity        String?
  entityId      String?
  details       String?
  createdAt     DateTime  @default(now())

  user          User?     @relation(fields: [userId], references: [id])
}

```

4.21-4.23 Treatment Plans

```
model TreatmentPlan {
  id          String    @id @default(cuid())
  patientId   String
  title       String
  description  String?
  status      String    @default("active") // active, completed, paused, cancelled
  startDate   DateTime  @default(now())
  endDate     DateTime?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  patient     Patient    @relation(fields: [patientId], references: [id], onDelete:
Cascade)
  phases      TreatmentPhase[]
}

model TreatmentPhase {
  id          String    @id @default(cuid())
  planId      String
  name        String
  description  String?
  order       Int       @default(0)
  status      String    @default("pending") // pending, in_progress, completed
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  plan        TreatmentPlan @relation(fields: [planId], references: [id], onDelete: Cascade)
  sessions    TreatmentPlanSession[]
}

model TreatmentPlanSession {
  id          String    @id @default(cuid())
  phaseId     String
  sessionId   String?
  notes       String?
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  phase       TreatmentPhase @relation(fields: [phaseId], references: [id], onDelete: Cascade)
  session     Session?       @relation(fields: [sessionId], references: [id])
}
```

4.24-4.26 Medications & Prescriptions

```
model Medication {
  id          String    @id @default(cuid())
  name        String
  category    String?   // cream, ointment, tablet, injection, lotion, gel
  description String?
  dosage      String?
  instructions String?
  sideEffects String?
  active       Boolean  @default(true)
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  prescriptions PrescriptionItem[]
}

model Prescription {
  id          String    @id @default(cuid())
  patientId   String
  doctorId    String?
  diagnosis   String?
  notes       String?
  date        DateTime  @default(now())
  createdAt   DateTime  @default(now())
  updatedAt   DateTime  @updatedAt

  patient     Patient    @relation(fields: [patientId], references: [id], onDelete: Cascade)
  doctor      User?      @relation(fields: [doctorId], references: [id])
  items       PrescriptionItem[]
}

model PrescriptionItem {
  id          String    @id @default(cuid())
  prescriptionId String
  medicationId String
  dosage      String?
  frequency   String?   // once daily, twice daily, etc.
  duration    String?   // 1 week, 2 weeks, etc.
  instructions String?
  createdAt   DateTime  @default(now())

  prescription Prescription @relation(fields: [prescriptionId], references: [id], onDelete: Cascade)
  medication    Medication  @relation(fields: [medicationId], references: [id])
}
```

4.27 Notification

```
model Notification {
  id          String    @id @default(cuid())
  userId      String?
  title       String
  message     String
  type        String    @default("info") // info, warning, success, error
  read        Boolean    @default(false)
  link        String?
  createdAt   DateTime  @default(now())
}
```

4.28 PartnerDoctor

```
model PartnerDoctor {
  id          String    @id @default(cuid())
  name        String
  phone       String?
  specialty   String?
  checkupPercentage Float @default(0) // نسبة الكشف %
  revisitPercentage Float @default(0) // نسبة الإعادة %
  laserPercentage Float @default(0) // نسبة الليزر %
  sessionPercentage Float @default(0) // نسبة الجلسات %
  fixedAmount Float @default(0) // مبلغ ثابت
  active      Boolean @default(true)
  notes       String?
  createdAt   DateTime @default(now())
  updatedAt   DateTime @updatedAt
}
```

4.29-4.30 Follow-up System

```
model FollowUpRecord {
  id                String    @id @default(cuid())
  patientId         String
  condition          String    // الحالة المزمنة / المرض
  conditionCategory String?    // جلدية، داخلية، أخرى
  severity           String    @default("moderate") // mild, moderate, severe, critical
  status             String    @default("active")   // active, paused, completed, discharged
  // Follow-up schedule
  frequency          String    @default("monthly") // weekly, biweekly, monthly, quarterly, custom
  customDays         Int?      // frequency = custom أيام مخصصة لو
  nextVisitDate      DateTime? // تاريخ الزيارة القادمة
  lastVisitDate      DateTime? // تاريخ آخر زيارة
  // Subscription / Package
  hasSubscription    Boolean    @default(false)     // عنده باقة متابعة؟
  subscriptionType   String?    // monthly, quarterly, yearly, session_based
  subscriptionPrice  Float      @default(0)         // سعر الباقة
  subscriptionStart  DateTime?   // بداية الباقة
  subscriptionEnd    DateTime?   // نهاية الباقة
  sessionsIncluded   Int        @default(0)         // عدد الجلسات في الباقة
  sessionsUsed       Int        @default(0)         // عدد الجلسات المستخدمة
  // Medical info
  diagnosis          String?    // التشخيص
  treatmentPlan      String?    // خطة العلاج
  medications        String?    // الأدوية الحالية (JSON string)
  notes              String?    // ملاحظات
  // Reminder
  reminderEnabled     Boolean    @default(true)     // تذكير تلقائي
  reminderDaysBefore Int        @default(1)         // كم يوم قبل الموعد
  createdAt           DateTime  @default(now())
  updatedAt           DateTime  @updatedAt

  patient             Patient    @relation(fields: [patientId], references: [id], onDelete:
Cascade)
  followUpVisits      FollowUpVisit[]
}

model FollowUpVisit {
  id                String    @id @default(cuid())
  followUpId        String
  visitNumber       Int        // رقم الزيارة
  // Visit details
  visitDate         DateTime  @default(now())
  type              String    @default("followup") // followup, checkup, emergency
  // Clinical findings
  findings           String?   // النتائج السريرية
  vitals             String?   // العلامات الحيوية (JSON)
  diagnosis          String?   // التشخيص
  treatmentNotes     String?   // ملاحظات العلاج
  medications        String?   // الأدوية الموصوفة (JSON string)
  instructions       String?   // التعليمات للمريض
  // Payment
  paid              Boolean    @default(false)
  price             Float      @default(0)
  // Next visit
  nextVisitDate      DateTime? // تاريخ الزيارة القادمة
  // Status
  status             String    @default("completed") // completed, no_show, cancelled
  notes              String?
  createdAt           DateTime  @default(now())
```

```
updatedAt      DateTime @updatedAt

followUp       FollowUpRecord @relation(fields: [followUpId], references: [id], onDelete: Cascade)
}
```

5. API Endpoints | واجهات البرمجة

Base URL: `/api`

All routes are in: `src/app/api/`

5.1 Authentication

Method	Path	Description
POST	<code>/api/auth/login</code>	Login (password: 2137)
GET	<code>/api/auth/me</code>	Get current user

5.2 Patients

Method	Path	Description
GET	<code>/api/patients</code>	List all patients (supports <code>?limit=</code>)
POST	<code>/api/patients</code>	Create patient
GET	<code>/api/patients/[id]</code>	Get patient by ID
PUT	<code>/api/patients/[id]</code>	Update patient
DELETE	<code>/api/patients/[id]</code>	Delete patient
POST	<code>/api/patients/import</code>	Import patients from JSON

5.3 Visits

Method	Path	Description
GET	<code>/api/visits</code>	List visits (supports <code>?patientId=</code> , <code>?doctorId=</code> , <code>?type=</code> , <code>?limit=</code>)
POST	<code>/api/visits</code>	Create visit
GET	<code>/api/visits/[id]</code>	Get visit
PUT	<code>/api/visits/[id]</code>	Update visit
DELETE	<code>/api/visits/[id]</code>	Delete visit

5.4 Sessions

Method	Path	Description
GET	/api/sessions	List sessions
POST	/api/sessions	Create session
GET	/api/sessions/[id]	Get session
PUT	/api/sessions/[id]	Update session
DELETE	/api/sessions/[id]	Delete session

5.5 Services

Method	Path	Description
GET	/api/services	List services
POST	/api/services	Create service
GET	/api/services/[id]	Get service
PUT	/api/services/[id]	Update service
DELETE	/api/services/[id]	Delete service

5.6 Finance

Method	Path	Description
GET	/api/finance/transactions	List transactions (supports ?type= , ?category= , ?startDate= , ?endDate= , ?limit=)
POST	/api/finance/transactions	Create transaction
GET	/api/finance/transactions/[id]	Get transaction
PUT	/api/finance/transactions/[id]	Update transaction
DELETE	/api/finance/transactions/[id]	Delete transaction
GET	/api/finance/summary	Financial summary

5.7 Laser

Method	Path	Description
GET	/api/laser/records	List laser records
POST	/api/laser/records	Create laser record
GET/PUT/DELETE	/api/laser/records/[id]	CRUD on record
GET/POST	/api/laser/sessions	List/Create laser sessions
GET/PUT/DELETE	/api/laser/sessions/[id]	CRUD on session
GET/POST	/api/laser/packages	List/Create packages
GET/PUT/DELETE	/api/laser/packages/[id]	CRUD on package
GET/POST	/api/laser/settings	List/Create settings
GET/PUT/DELETE	/api/laser/settings/[id]	CRUD on setting
GET/POST	/api/laser/notes	List/Create notes
GET/PUT/DELETE	/api/laser/notes/[id]	CRUD on note

5.8 Follow-up

Method	Path	Description
GET/POST	/api/follow-up/records	List/Create follow-up records
GET/PUT/DELETE	/api/follow-up/records/[id]	CRUD on record
GET/POST	/api/follow-up/visits	List/Create follow-up visits
GET/PUT/DELETE	/api/follow-up/visits/[id]	CRUD on visit

5.9 Other Endpoints

Method	Path	Description
GET/POST/PUT/DELETE	/api/notes	Notes CRUD
GET/POST/PUT/DELETE	/api/alerts	Alerts CRUD
GET/POST/PUT/DELETE	/api/reminders	Reminders CRUD
GET/POST/PUT/DELETE	/api/appointments	Appointments CRUD
GET/POST/PUT/DELETE	/api/waiting	Waiting queue CRUD
GET/POST/PUT/DELETE	/api/doctors	Partner doctors CRUD
GET/POST/PUT/DELETE	/api/inventory/items	Inventory items CRUD
GET/POST/PUT/DELETE	/api/inventory/transactions	Inventory transactions CRUD
GET/POST/PUT/DELETE	/api/medications	Medications CRUD
GET/POST/PUT/DELETE	/api/prescriptions	Prescriptions CRUD
GET/POST/PUT/DELETE	/api/treatment-plans	Treatment plans CRUD
GET/POST/PUT/DELETE	/api/treatment-plans/[id]/phases	Plan phases CRUD
GET/PUT/DELETE	/api/treatment-plans/phases/[phaseId]	Phase CRUD
GET/POST/PUT/DELETE	/api/notifications	Notifications CRUD
GET/POST/PUT/DELETE	/api/photos	Patient photos CRUD
GET/POST	/api/backups	List/Create backups
POST	/api/backups/import	Restore from JSON
GET/DELETE	/api/backups/[id]	Get/Delete backup
GET	/api/reports/daily	Daily report
GET	/api/reports/weekly	Weekly report
GET	/api/reports/monthly	Monthly report
POST	/api/ai/chat	AI chat endpoint
GET	/api/audit-logs	Audit logs
POST	/api/seed	Seed initial data
POST	/api/clear-data	Clear all data

API Conventions:

Response format: { data: [...] } or { patients/visits/sessions/etc: [...] }

Pagination: { pagination: { page, limit, total, pages } }

Date handling: All dates use `toCairoDate()` from `@/lib/cairo-time`

Error format: `{ error: "message" }` with appropriate HTTP status

6. Key Code Functions | الدوال المهمة

.All these functions are in `src/app/page.tsx` unless otherwise noted

6.1 `normalizePhone(phone: string): string`

Location: `src/app/page.tsx` line ~75 **Purpose:** Convert Arabic/Persian numerals and symbols to Latin equivalents

Critical for: WhatsApp integration — Arabic digits cause "not a valid phone number" error

```
const normalizePhone = (phone: string): string => {
  return phone
  // Convert Arabic-Indic numerals (٠-٩) used in Egypt/Arabic
  .replace(/[٠-٩]/g, d => '٠١٢٣٤٥٦٧٨٩'.indexOf(d).toString())
  // Convert Persian/Urdu numerals (۰-۹) used in some keyboards
  .replace(/[۰-۹]/g, d => '٠١٢٣٤٥٦٧٨٩'.indexOf(d).toString())
  // Convert Arabic full-width plus sign to Latin +
  .replace(/+/g, '+')
  // Convert Arabic comma/dash/space to Latin equivalents
  .replace(/[,.'/]/g, ',')
  .replace(/_/g, '-')
  // Remove common RTL/LTR marks and Arabic tatweel
  .replace(/[\u200E\u200F\u200C\u200D\u0640]/g, '')
}
```

6.2 `waPhone(phone?: string): string`

Location: `src/app/page.tsx` line ~91 **Purpose:** Format phone for WhatsApp (adds Egypt country code +20 if missing) **Used in:** All WhatsApp links `https://wa.me/${waPhone(patient.phone)}`

```
const waPhone = (phone?: string) => {
  if (!phone) return ''
  // Step 1: Normalize ALL Arabic/Persian numerals to Latin
  const normalized = normalizePhone(phone)
  // Step 2: Extract only digits
  const digits = normalized.replace(/^[^0-9]/g, '')
  if (!digits || digits.length < 3) return '' // too short to be valid
  // Step 3: Add Egypt country code if missing
  if (digits.startsWith('20')) return digits // already has country code
  if (digits.startsWith('0')) return '2' + digits // starts with 0 → add 2
  return '20' + digits // no prefix → add 20
}
```

6.3 `getVisitCategory(type: string): string`

Location: `src/app/page.tsx` line ~355 **Purpose:** Map visit type to financial category — SINGLE SOURCE OF TRUTH **Critical for:** Correct financial recording (جلسات vs إعادة vs كشف)

```
const getVisitCategory = (type: string): string => {
  switch (type) {
    case 'checkup': return 'كشف'
    case 'revisit': return 'إعادة'
    case 'session': return 'جلسات'
    case 'checkup_session': return 'كشف'
    case 'revisit_session': return 'إعادة'
    default: return 'كشف'
  }
}
```

6.4 `cairoDateTime(dateStr?: string): string | undefined`

Location: `src/app/page.tsx` line ~173 **Purpose:** Combine a date (YYYY-MM-DD) with current Cairo time → "YYYY-MM-DDTHH:MM:00" **Critical for:** Fixes 4-hour timestamp offset — bare "YYYY-MM-DD" was interpreted as midnight

```
const cairoDateTime = (dateStr?: string): string | undefined => {
  if (!dateStr) return undefined
  return `${dateStr}T${cairoTimeInput()}:00`
}
```

6.5 `cairoTodayInput(): string`

Location: `src/app/page.tsx` line ~160 **Purpose:** Get today's date in Cairo timezone as YYYY-MM-DD (for date input defaults)

```
const cairoTodayInput = () => new Date().toLocaleDateString('en-CA', { timeZone: 'Africa/Cairo' })
```

6.6 `cairoTimeInput(): string`

Location: `src/app/page.tsx` line ~163 **Purpose:** Get current time in Cairo as HH:MM

```
const cairoTimeInput = () => {
  const now = new Date()
  return now.toLocaleTimeString('en-GB', { timeZone: 'Africa/Cairo', hour: '2-digit', minute: '2-digit' })
}
```

6.7 `cairoISO(): string`

Location: `src/app/page.tsx` line ~150 **Purpose:** Get current Cairo time as ISO string (UTC)

```
const cairoISO = () => {
  const now = new Date()
  return now.toISOString()
}
```

6.8 `getLocalDateStr(date?: Date | string): string`

Location: `src/app/page.tsx` line ~105 **Purpose:** Get local date string in Cairo timezone (fixes UTC offset issue)

```
const getLocalDateStr = (date?: Date | string) => {
  const d = date ? new Date(date) : new Date()
  return d.toLocaleDateString('en-CA', { timeZone: 'Africa/Cairo' })
}
```

6.9 normalizeArabic(text: string): string

Location: `src/app/page.tsx` line ~308 **Purpose:** Arabic text normalization for search (remove diacritics, normalize alef/yaa/taa)

```
const normalizeArabic = (text: string): string => {
  return text
    .replace(/[اآإئ]/g, 'ا') // normalize alef variants
    .replace(/[ة]/g, 'ه') // taa marbuta → haa
    .replace(/[ى]/g, 'ي') // alef maqsurah → yaa
    .replace(/[\u064B-\u065F\u0670]/g, '') // remove diacritics/tashkeel
    .replace(/\s+/g, ' ') // normalize whitespace
    .trim()
    .toLowerCase()
}
```

6.10 fuzzyMatch(query: string, target: string): boolean

Location: `src/app/page.tsx` line ~320 **Purpose:** Fuzzy match — checks if query characters appear in order in target

```
const fuzzyMatch = (query: string, target: string): boolean => {
  const nq = normalizeArabic(query)
  const nt = normalizeArabic(target)
  if (nt.includes(nq)) return true // direct substring match (normalized)
  // Character-by-character fuzzy match
  let qi = 0
  for (let ti = 0; ti < nt.length && qi < nq.length; ti++) {
    if (nt[ti] === nq[qi]) qi++
  }
  return qi === nq.length
}
```

6.11 smartSearch(query, fields): { match, score }

Location: `src/app/page.tsx` line ~333 **Purpose:** Smart search — tries exact, then normalized, then fuzzy

```
const smartSearch = (query: string, fields: (string | undefined)[]): { match: boolean; score: number }
=> {
  if (!query.trim()) return { match: false, score: 0 }
  const nq = normalizeArabic(query)
  let bestScore = 0
  for (const field of fields) {
    if (!field) continue
    const nf = normalizeArabic(field)
    // Exact substring (highest score)
    if (nf.includes(nq)) {
      const score = nq.length / nf.length + 1
      if (score > bestScore) bestScore = score
    }
    // Fuzzy match (lower score)
    else if (fuzzyMatch(query, field)) {
      const score = 0.5
      if (score > bestScore) bestScore = score
    }
  }
  return { match: bestScore > 0, score: bestScore }
}
```

6.12 `apiFetch<T>(path, options?): Promise<T>`

Location: `src/app/page.tsx` line ~178 **Purpose:** Fetch wrapper for API calls with error handling

```
async function apiFetch<T>(path: string, options?: RequestInit): Promise<T> {
  const res = await fetch(`/api${path}`, { headers: { 'Content-Type': 'application/json' }, ...options })
  if (!res.ok) {
    const text = await res.text().catch(() => '')
    try {
      const errData = JSON.parse(text)
      throw new Error(errData.error || errData.details || JSON.stringify(errData) || `Error ${res.status}`)
    } catch (parseErr) {
      if (parseErr instanceof Error && parseErr.message && !parseErr.message.includes('JSON')) throw parseErr
      throw new Error(text || `Error ${res.status}`)
    }
  }
  if (res.status === 204) return undefined as T
  return res.json()
}
```

6.13 `addItem<T>(path, body, setter, silent?): Promise<T>`

Location: `src/app/page.tsx` line ~1118 **Purpose:** POST helper that adds item to state and shows toast **Important:** `silent` parameter added to prevent duplicate toasts

```
const addItem = async <T,>(path: string, body: any, setter: React.Dispatch<React.SetStateAction<T[]>>,
silent = false) => {
  try {
    const res = await apiFetch<any>(path, { method: 'POST', body: JSON.stringify(body) })
    const item = res?.data || res?.patient || res?.visit || res?.session || res?.service || res?.note
    || res?.alert || res?.reminder || res?.record || res?.package || res?.setting || res?.transaction ||
    res?.appointment || res?.item || res?.plan || res?.medication || res?.prescription || res?.backup ||
    res
    if (item?.id) setter(prev => [item, ...prev])
    if (!silent) toast.success('تمت الإضافة بنجاح')
    return item
  } catch (e: any) {
    if (!silent) toast.error(e.message || 'خطأ')
    return null
  }
}
```

6.14 deleteItem<T>(path, id, setter): Promise<void>

Location: `src/app/page.tsx` line ~1121

```
const deleteItem = async <T,>(path: string, id: string, setter:
React.Dispatch<React.SetStateAction<T[]>>) => {
  try {
    await apiFetch(`${path}/${id}`, { method: 'DELETE' })
    setter(prev => prev.filter((item: any) => item.id !== id))
    toast.success('تم الحذف')
  } catch (e: any) {
    toast.error(e.message || 'خطأ')
  }
}
```

6.15 Server-side: toCairoDate(dateStr?): Date

Location: `src/lib/cairo-time.ts` line ~141 Purpose: Parse a date string as Cairo time and return a UTC Date

Critical: Handles DST, timezone offsets, and date-only vs datetime strings

```
export function toCairoDate(dateStr?: string): Date {
  if (!dateStr) {
    return new Date() // current moment in UTC
  }
  if (dateStr.includes('+') || dateStr.includes('Z')) {
    return new Date(dateStr) // already has timezone info
  }
  if (dateStr.includes('T')) {
    // Has time but no timezone – interpret as Cairo local time
    const datePart = dateStr.slice(0, 10)
    const offsetMinutes = getCairoOffsetMinutes(new Date(datePart + 'T12:00:00'))
    const asUtc = new Date(dateStr)
    return new Date(asUtc.getTime() - offsetMinutes * 60 * 1000)
  }
  // Date-only string – interpret as Cairo midnight
  return cairoDayRange(dateStr).gte
}
```

6.16 Server-side: `cairoDayRange(dateStr?): { gte: Date; lt: Date }`

Location: `src/lib/cairo-time.ts` line ~50 **Purpose:** Get Cairo start-of-day and end-of-day as UTC Date objects (for Prisma queries)

```
export function cairoDayRange(dateStr?: string): { gte: Date; lt: Date } {
  const today = dateStr || cairoTodayStr()
  const offsetMinutes = getCairoOffsetMinutes(new Date(today + 'T12:00:00'))
  const [y, m, d] = today.split('-').map(Number)
  const utcMidnight = new Date(Date.UTC(y, m - 1, d, 0, 0, 0))
  const utcStart = new Date(utcMidnight.getTime() - offsetMinutes * 60 * 1000)
  const utcEnd = new Date(utcStart.getTime() + 24 * 60 * 60 * 1000)
  return { gte: utcStart, lt: utcEnd }
}
```

6.17 VISIT_TYPES Constant

Location: `src/app/page.tsx` line ~367

```
const VISIT_TYPES = [
  { id: 'checkup', label: 'كشف', emoji: '🩺', bg: 'bg-emerald-500', hoverBg: 'hover:bg-emerald-600',
    ring: 'ring-emerald-300' },
  { id: 'revisit', label: 'إعادة', emoji: '🔄', bg: 'bg-blue-500', hoverBg: 'hover:bg-blue-600', ring:
    'ring-blue-300' },
  { id: 'session', label: 'جلسة', emoji: '⌚', bg: 'bg-violet-500', hoverBg: 'hover:bg-violet-600',
    ring: 'ring-violet-300' },
  { id: 'checkup_session', label: 'كشف + جلسة', emoji: '🩺⌚', bg: 'bg-gradient-to-l from-emerald-500
    to-violet-500', hoverBg: 'hover:from-emerald-600 hover:to-violet-600', ring: 'ring-emerald-300' },
  { id: 'revisit_session', label: 'إعادة + جلسة', emoji: '🔄⌚', bg: 'bg-gradient-to-l from-blue-500 to-
    violet-500', hoverBg: 'hover:from-blue-600 hover:to-violet-600', ring: 'ring-blue-300' },
]
```

6.18 Default Prices (in patient profile visit form)

كشف (checkup): 200 EGP

إعادة (revisit): 80 EGP

جلسة (session): manual entry (no default)

6.19 Financial Categories for Transactions

كشف — consultation

إعادة — revisit

جلسات — sessions

ليزر — laser

متابعة — follow-up

personal — personal (excluded from clinic revenue)

clinic — default clinic category

7. File Structure | هيكل الملفات

```
my-project/
├── prisma/
│   └── schema.prisma           # Database schema (30 models)
├── public/
│   ├── favicon.ico
│   ├── logo.svg
│   └── robots.txt
├── src/
│   ├── app/
│   │   ├── api/               # All API routes (60+ files)
│   │   │   ├── auth/login/
│   │   │   ├── auth/me/
│   │   │   ├── patients/
│   │   │   ├── visits/
│   │   │   ├── sessions/
│   │   │   ├── services/
│   │   │   ├── finance/transactions/
│   │   │   ├── finance/summary/
│   │   │   ├── laser/records/
│   │   │   ├── laser/sessions/
│   │   │   ├── laser/packages/
│   │   │   ├── laser/settings/
│   │   │   ├── laser/notes/
│   │   │   ├── follow-up/records/
│   │   │   ├── follow-up/visits/
│   │   │   ├── notes/
│   │   │   ├── alerts/
│   │   │   ├── reminders/
│   │   │   ├── appointments/
│   │   │   ├── waiting/
│   │   │   ├── doctors/
│   │   │   ├── inventory/items/
│   │   │   ├── inventory/transactions/
│   │   │   ├── medications/
│   │   │   ├── prescriptions/
│   │   │   ├── treatment-plans/
│   │   │   ├── notifications/
│   │   │   ├── photos/
│   │   │   ├── backups/
│   │   │   ├── backups/import/
│   │   │   ├── reports/daily/
│   │   │   ├── reports/weekly/
│   │   │   ├── reports/monthly/
│   │   │   ├── ai/chat/
│   │   │   ├── audit-logs/
│   │   │   ├── seed/
│   │   │   └── clear-data/
│   │   ├── globals.css        # Global styles + luxury theme (1167+ lines)
│   │   ├── layout.tsx         # Root layout (RTL, fonts, ThemeProvider)
│   │   └── page.tsx           # MAIN APP – monolithic (~7300+ lines)
│   └── components/
│       └── ui/                # shadcn/ui components (48 files)
│           ├── accordion.tsx
│           ├── alert-dialog.tsx
│           ├── alert.tsx
│           ├── aspect-ratio.tsx
│           ├── avatar.tsx
│           ├── badge.tsx
│           └── breadcrumb.tsx
```



```

├── button.tsx
├── calendar.tsx
├── card.tsx
├── carousel.tsx
├── chart.tsx
├── checkbox.tsx
├── collapsible.tsx
├── command.tsx
├── context-menu.tsx
├── dialog.tsx
├── drawer.tsx
├── dropdown-menu.tsx
├── form.tsx
├── hover-card.tsx
├── input-otp.tsx
├── input.tsx
├── label.tsx
├── menubar.tsx
├── navigation-menu.tsx
├── pagination.tsx
├── popover.tsx
├── progress.tsx
├── radio-group.tsx
├── resizable.tsx
├── scroll-area.tsx
├── select.tsx
├── separator.tsx
├── sheet.tsx
├── sidebar.tsx
├── skeleton.tsx
├── slider.tsx
├── sonner.tsx
├── switch.tsx
├── table.tsx
├── tabs.tsx
├── textarea.tsx
├── toast.tsx
├── toaster.tsx
├── toggle-group.tsx
├── toggle.tsx
├── tooltip.tsx
├── hooks/
│   ├── use-mobile.ts
│   └── use-toast.ts
├── lib/
│   ├── api.ts                # API utilities
│   ├── cairo-time.ts         # Cairo timezone helpers (CRITICAL)
│   ├── db.ts                 # Prisma client singleton
│   ├── store.ts              # Zustand stores (auth + clinic settings)
│   └── utils.ts               # cn(), formatCurrency(), formatDate(), formatTime()
├── .env.example               # Environment variable template
├── .gitignore
├── bun.lock
├── Caddyfile
├── components.json            # shadcn/ui config
├── eslint.config.mjs
├── next.config.ts
├── next-env.d.ts
├── package-lock.json
├── package.json
├── postcss.config.mjs

```

```
└─ tailwind.config.ts
└─ tsconfig.json
```

8. Environment Variables | المتغيرات البيئية

Required:

```
# Database URL – Neon PostgreSQL (production)
DATABASE_URL=postgresql://user:password@ep-xxx.region.aws.neon.tech/neondb?sslmode=require
```

How to get DATABASE_URL:

Go to <https://neon.tech>

Create account → Create new project

Copy connection string from dashboard

Format: `postgresql://USER:PASSWORD@ep-HOSTNAME.REGION.aws.neon.tech/DBNAME?sslmode=require`

Vercel Environment Variables (set in Vercel dashboard):

`DATABASE_URL` — Neon connection string

No other env vars needed! The app uses:

Simple password auth (hardcoded `2137` in `/api/auth/login/route.ts`)

No external API keys (AI uses `z-ai-web-dev-sdk` which is pre-configured)

No SMTP/email services

No payment gateways

9. Configuration Files | ملفات الإعدادات

9.1 next.config.ts

```
import type { NextConfig } from "next";

const nextConfig: NextConfig = {
  typescript: {
    ignoreBuildErrors: true,
  },
  reactStrictMode: false,
  experimental: {
    serverActions: {
      bodySizeLimit: '100mb', // For large backup imports
    },
  },
};

export default nextConfig;
```

9.2 tailwind.config.ts

```
import type { Config } from "tailwindcss";
import tailwindcssAnimate from "tailwindcss-animate";

const config: Config = {
  darkMode: "class",
  content: [
    "./pages/**/*..{js,ts,jsx,tsx,mdx}",
    "./components/**/*..{js,ts,jsx,tsx,mdx}",
    "./app/**/*..{js,ts,jsx,tsx,mdx}",
  ],
  theme: {
    extend: {
      colors: {
        background: 'hsl(var(--background))',
        foreground: 'hsl(var(--foreground))',
        card: { DEFAULT: 'hsl(var(--card))', foreground: 'hsl(var(--card-foreground))' },
        popover: { DEFAULT: 'hsl(var(--popover))', foreground: 'hsl(var(--popover-foreground))' },
        primary: { DEFAULT: 'hsl(var(--primary))', foreground: 'hsl(var(--primary-foreground))' },
        secondary: { DEFAULT: 'hsl(var(--secondary))', foreground: 'hsl(var(--secondary-foreground))' },
      },
      muted: { DEFAULT: 'hsl(var(--muted))', foreground: 'hsl(var(--muted-foreground))' },
      accent: { DEFAULT: 'hsl(var(--accent))', foreground: 'hsl(var(--accent-foreground))' },
      destructive: { DEFAULT: 'hsl(var(--destructive))', foreground: 'hsl(var(--destructive-foreground))' },
      border: 'hsl(var(--border))',
      input: 'hsl(var(--input))',
      ring: 'hsl(var(--ring))',
      chart: {
        '1': 'hsl(var(--chart-1))', '2': 'hsl(var(--chart-2))',
        '3': 'hsl(var(--chart-3))', '4': 'hsl(var(--chart-4))', '5': 'hsl(var(--chart-5))'
      }
    },
    borderRadius: {
      lg: 'var(--radius)', md: 'calc(var(--radius) - 2px)', sm: 'calc(var(--radius) - 4px)'
    }
  },
  plugins: [tailwindcssAnimate],
};
export default config;
```

9.3 src/app/layout.tsx

```
import type { Metadata } from "next";
import { Cairo, Playfair_Display } from "next/font/google";
import { ThemeProvider } from "next-themes";
import { Toaster } from "@components/ui/sonner";
import "../globals.css";

const cairo = Cairo({
  subsets: ["arabic", "latin"],
  variable: "--font-cairo",
  display: "swap",
});

const playfair = Playfair_Display({
  subsets: ["latin"],
  variable: "--font-playfair",
  display: "swap",
});

export const metadata: Metadata = {
  title: "Elmoghazi Clinic | عيادة المغازي",
  description: "نظام إدارة العيادة الجلدية",
  icons: { icon: "/favicon.ico" },
};

export default function RootLayout({ children }: Readonly<{ children: React.ReactNode }>) {
  return (
    <html lang="ar" dir="rtl" suppressHydrationWarning>
      <body className={` ${cairo.variable} ${playfair.variable} font-cairo antialiased bg-background text-foreground`} >
        <ThemeProvider attribute="class" defaultTheme="light" enableSystem disableTransitionOnChange>
          {children}
          <Toaster position="top-left" richColors />
        </ThemeProvider>
      </body>
    </html>
  );
}
```

9.4 src/lib/db.ts

```
import { PrismaClient } from '@prisma/client'

const globalForPrisma = globalThis as unknown as { prisma: PrismaClient }

export const db = globalForPrisma.prisma || new PrismaClient()

if (process.env.NODE_ENV !== 'production') globalForPrisma.prisma = db
```

9.5 `src/lib/utils.ts`

```
import { type ClassValue, clsx } from "clsx"
import { twMerge } from "tailwind-merge"

export function cn(...inputs: ClassValue[]) {
  return twMerge(clsx(inputs))
}

export function safeName(name: string | null | undefined): string {
  if (!name) return 'بدون اسم'
  return name.trim() || 'بدون اسم'
}

export function formatCurrency(amount: number): string {
  return `${amount.toLocaleString('ar-EG')} ج.م`
}

export function formatDate(date: string | Date): string {
  return new Date(date).toLocaleDateString('ar-EG', { year: 'numeric', month: 'long', day: 'numeric',
    timeZone: 'Africa/Cairo' })
}

export function formatTime(date: string | Date): string {
  return new Date(date).toLocaleTimeString('ar-EG', { hour: '2-digit', minute: '2-digit', timeZone:
    'Africa/Cairo' })
}

export function generateFileNumber(): string {
  const chars = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ'
  const random = Array.from({ length: 6 }, () => chars[Math.floor(Math.random() *
    chars.length)])
  return `F-${random}`
}
```

9.6 `src/lib/store.ts` (Zustand stores)

`useAuthStore` — persisted as `elmoghazi-auth` in localStorage

`user`, `isAuthenticated`, `userRole` ('doctor' | 'secretary')

`login()`, `logout()`, `setUserRole()`

`useClinicStore` — persisted as `elmoghazi-clinic-settings`

`theme` (10 themes), `activeTab`, `statusColors`, `autoBackup`, `backupInterval`, `lastBackup`,
`sectionPasswords`

`THEME_CONFIGS` — 10 theme configurations:

emerald, royal, rosegold, purple, sunset, teal, ruby, forest, midnight, champagne

10. UI/UX Design Notes | ملاحظات التصميم

Colors & Themes:

Default Theme: Emerald (زمردی) — `#047857` primary, `#D4A843` accent

10 Available Themes: emerald, royal, rosegold, purple, sunset, teal, ruby, forest, midnight, champagne

Dark Mode: Full support via `class="dark"` on `<html>`

CSS Variables: HSL-based tokens in `globals.css`

Typography:

Arabic Font: Cairo (from Google Fonts)

Latin Font: Playfair Display (from Google Fonts)

Fallback: `'Noto Sans SC', 'Segoe UI', sans-serif`

Direction: RTL (`dir="rtl"` on `<html>`)

Luxury Design Elements:

Gradient backgrounds (`bg-gradient-to-br` , `bg-gradient-to-l`)

Glassmorphism (`backdrop-blur-sm` , `bg-white/20`)

Soft shadows (`shadow-lg` , `shadow-xl` , `shadow-2xl`)

Smooth animations (Framer Motion — hover, tap, page transitions)

Glow effects (`blur-3xl` colored backgrounds)

Rounded corners (`rounded-xl` , `rounded-2xl` , `rounded-3xl`)

Animated emojis (rotating, scaling, bouncing)

Component Patterns:

`card-luxury` class for premium cards

`input-luxury` class for premium inputs

`btn-luxury` class for premium buttons

`section-header-animated` for section headers

Gradient buttons with hover effects

Motion buttons with `whileTap={{ scale: 0.9 }}` and `whileHover={{ scale: 1.05 }}`

Responsive Breakpoints:

Mobile-first design

Grid: `grid-cols-2 sm:grid-cols-4 md:grid-cols-6`

Navigation: bottom nav on mobile, full nav on desktop

Icons (Lucide React):

Used extensively: LayoutDashboard, Users, Stethoscope, Zap, Search, Bell, Moon, Sun, Plus, Edit3, Trash2, Star, Phone, Calendar, Clock, DollarSign, Package, FileText, Activity, AlertTriangle, CheckCircle, ChevronDown, Settings, Shield, BarChart3, TrendingUp, Eye, Camera, Pill, Heart, Send, Bot, RefreshCw, Download, Upload, Filter, UserPlus, Sparkles, Hash, MapPin, Palette, X, Database, HardDrive, Archive, FileDown, FileUp, Timer, Tag, Scissors, Syringe, Layers, Wand2, ThermometerSun, Lock, CircleDot, Armchair, ScanFace, Hand, Circle, MousePointerClick, Target, ZapOff, BarChart2, Receipt, CalendarCheck, UsersRound, ClipboardCheck, AlertCircle, Wallet, TrendingDown,

11. Bug Fixes Applied | الإصلاحات المطبقة

.These are critical fixes that must be preserved in any rebuild

Fix 1: WhatsApp Arabic Digits (commit 09a884c)

Problem: Phone numbers entered with Arabic digits (٠١٢٣) caused "Is not a valid phone number" error in WhatsApp.

Solution: Added `normalizePhone()` function that converts:

Arabic-Indic numerals (٠-٩) → Latin (0-9)

Persian numerals (۰-۹) → Latin (0-9)

Arabic plus (+) → Latin (+)

RTL/LTR marks → removed

Applied to: `waPhone()` function AND all phone input fields (`onChange` uses `normalizePhone`)

Fix 2: Visit Type → Financial Category (commit f21325c)

Problem: When selecting "إعادة" (80 EGP) from patient file, it recorded as "كشف" (200 EGP) in financial ledger.

Solution: Created `getVisitCategory()` as SINGLE SOURCE OF TRUTH:

```
const getVisitCategory = (type: string): string => {
  switch (type) {
    case 'checkup': return 'كشف'
    case 'revisit': return 'إعادة'
    case 'session': return 'جلسات'
    case 'checkup_session': return 'كشف'
    case 'revisit_session': return 'إعادة'
    default: return 'كشف'
  }
}
```

Replaced all inline ternary logic with this function.

Fix 3: Robust Visit Type/Price Handling (commit 2968963)

Problem: Stale closure issues and duplicate toasts when saving visits from patient profile. **Solution:**

Capture state values into local variables BEFORE any `await` calls

Added `silent` parameter to `addItem()` to prevent duplicate toasts

Auto-set price when visit type is selected (80=إعادة, 200=كشف)

Single toast at end with category and price info: تم إضافة الزيارة - إعادة 80 ج.م

Fix 4: Timestamp Offset — 4 Hour Difference (commit [6176dec](#))

Problem: Visits/sessions recorded with ~4 hour offset from actual entry time. **Root Cause:** Date input returns "YYYY-MM-DD" (date-only). API's `toCairoDate()` interprets this as **Cairo midnight (00:00)**, causing a multi-hour offset vs. the actual entry time (e.g., 4 PM). **Solution:** Created `cairoDateTime()` helper:

```
const cairoDateTime = (dateStr?: string): string | undefined => {
  if (!dateStr) return undefined
  return `${dateStr}T${cairoTimeInput()}:00` // "2026-06-16T16:07:00"
}
```

Applied to ALL places that send dates to the API:

- ✓ Patient file visit save handler
- ✓ Patient file session save handler
- ✓ New patient form (visit + session creation)
- ✓ Manual transaction form

Fix 5: Cairo Timezone Throughout

Problem: Vercel servers run in UTC, causing date/time mismatches with Cairo (UTC+2/+3 DST). **Solution:** Comprehensive `src/lib/cairo-time.ts` with:

- ✓ `cairoNow()` — current UTC moment
- ✓ `cairoTodayStr()` — today's date in Cairo
- ✓ `cairoDayRange(dateStr)` — start/end of Cairo day as UTC (for Prisma queries)
- ✓ `cairoWeekRange()` — Saturday to Friday week range (Egyptian week)
- ✓ `cairoMonthRange()` — month range
- ✓ `toCairoDate(dateStr)` — parse date string as Cairo time → UTC Date
- ✓ `getCairoOffsetMinutes(date)` — handles DST automatically

Fix 6: Backup/Restore Virtual Fields

Problem: Restore from backup failed due to virtual/relation fields in JSON. **Solution:** `src/app/api/backups/import/route.ts` strips virtual fields before Prisma createMany:

```
const VIRTUAL_FIELDS = [
  '_count', 'patient', 'doctor', 'user', 'service', 'laserRecord',
  'laserPackage', 'inventoryItem', 'treatmentPlan', 'phase',
  'visits', 'sessions', 'alerts', 'patientNotes', 'laserRecords',
  'appointments', 'photos', 'treatmentPlans', 'reminders',
  'waitingQueue', 'prescriptions', 'followUpRecords', 'followUpVisits',
  'items', 'transactions', 'notes', 'medications',
]
```

Fix 7: Git History Secret Cleanup

Problem: Vercel PAT token was in git history via `.env.production`. **Solution:** Created orphan branch with clean history and force pushed. `.env.production` added to `.gitignore`.

12. Deployment Guide | دليل النشر

Option A: Deploy on Vercel (recommended)

Step 1: Prerequisites

GitHub account

Vercel account (<https://vercel.com>)

Neon account (<https://neon.tech>) — for PostgreSQL

Step 2: Database Setup (Neon)

Sign up at <https://neon.tech>

Create new project → name it "elmoghazi-clinic"

Select region: AWS ap-southeast-1 (Singapore) or closest to your users

Copy the connection string:

```
postgresql://USER:PASSWORD@ep-HOSTNAME.REGION.aws.neon.tech/DBNAME?sslmode=require
```

Save this — you'll need it for Vercel

Step 3: Deploy on Vercel

Go to <https://vercel.com/new>

Import your GitHub repository: `sayedhassan589-art/elmoghazi-clinic`

Configure project:

Framework Preset: Next.js

Build Command: `npm run build` (default)

Install Command: `npm install` (default)

Environment Variables:

Add `DATABASE_URL` = your Neon connection string

Click "Deploy"

Wait for build to complete (~2-3 minutes)

Step 4: Initialize Database

After deployment, visit: <https://YOUR-APP.vercel.app/api/seed>

This creates initial users (doctor@elmoghazi.com, secretary@elmoghazi.com)

The app is now ready! Login with password: `2137`

Step 5: Custom Domain (optional)

In Vercel dashboard → Settings → Domains

Add your custom domain

Configure DNS as instructed

Option B: Deploy on Other Platforms

Netlify:

Connect GitHub repo

Build command: `npm run build`

Publish directory: `.next`

Add `DATABASE_URL` env var

Note: Netlify has some Next.js compatibility issues — Vercel is recommended

Railway:

Connect GitHub repo

Add `DATABASE_URL` env var

Railway auto-detects Next.js

Deploy

Render:

Create new Web Service from GitHub

Build command: `npm install && npm run build`

Start command: `npm start`

Add `DATABASE_URL` env var

Self-hosted (VPS):

Clone repo: `git clone https://github.com/sayedhassan589-art/elmoghazi-clinic.git`

Install dependencies: `npm install`

Create `.env` file with `DATABASE_URL`

Build: `npm run build`

Start: `npm start`

Use PM2 for process management: `pm2 start npm --name "clinic" -- start`

Use Nginx as reverse proxy

Database Migration:

When deploying to a new database, run:

```
npx prisma db push
```

This creates all tables based on `schema.prisma`.

13. Recovery Guide | دليل الاستعادة من الكوارث

If the app is lost, here's how to rebuild:

Scenario 1: Code is on GitHub, database is lost

Deploy code from GitHub to Vercel (see Deployment Guide)

Create new Neon database

Update `DATABASE_URL` in Vercel

Run `npx prisma db push` (or visit `/api/seed`)

If you have a backup JSON file:

Login to the app

Go to: المزيد → النسخ الاحتياطي → استعادة

Upload the JSON file

All data will be restored

Scenario 2: Everything is lost (no code, no database)

Give this specification file to a developer

Developer follows these steps:

- Create new Next.js 16 project with TypeScript
- Install all dependencies (see Section 2)
- Set up Prisma with the schema (see Section 4)
- Create all API routes (see Section 5)
- Create the main `page.tsx` with all features (see Section 3 & 6)
- Set up Cairo timezone helpers (see Section 6.15-6.16)
- Apply all bug fixes (see Section 11)
- Deploy (see Section 12)

Scenario 3: Moving to a new platform

Export full backup from current app:

Go to: المزيد → النسخ الاحتياطي → تصدير

Download the JSON file

Deploy code on new platform (see Section 12)

Update `DATABASE_URL` to new database

Run `npx prisma db push`

Import the backup JSON:

Login to new app

Go to: المزيد → النسخ الاحتياطي → استعادة

Upload the JSON file

Backup File Format:

```
{
  "exportedAt": "2026-06-16T13:00:00.000Z",
  "version": "1.0",
  "data": {
    "users": [...],
    "patients": [...],
    "services": [...],
    "visits": [...],
    "sessions": [...],
    "notes": [...],
    "alerts": [...],
    "reminders": [...],
    "laserRecords": [...],
    "laserSessions": [...],
    "laserPackages": [...],
    "laserSettings": [...],
    "laserNotes": [...],
    "transactions": [...],
    "appointments": [...],
    "waitingQueue": [...],
    "inventoryItems": [...],
    "inventoryTransactions": [...],
    "treatmentPlans": [...],
    "treatmentPhases": [...],
    "treatmentPlanSessions": [...],
    "patientPhotos": [...],
    "medications": [...],
    "prescriptions": [...],
    "prescriptionItems": [...],
    "notifications": [...],
    "auditLogs": [...],
    "partnerDoctors": [...],
    "followUpRecords": [...],
    "followUpVisits": [...]
  }
}
```

Backup Best Practices:

Weekly full backup: Download JSON file every week

Before updates: Always backup before deploying new code

Store in 2 places: Local computer + cloud (Google Drive, Dropbox)

Test restore: Periodically test restoring to verify backup integrity

Neon branching: Use Neon's branching feature for testing

14. Login & Auth | تسجيل الدخول والصلاحيات

Login Flow:

User opens app → sees login screen

Selects role: طبيب (doctor) or سكرتيرة (secretary)

Enters password: 2137

POST /api/auth/login with { role, password }

Server verifies password (hardcoded 2137 in /api/auth/login/route.ts)

Auto-creates user if doesn't exist:

doctor@elmoghazi.com (for doctor role)

secretary@elmoghazi.com (for secretary role)

Returns user object (without password)

Frontend stores in Zustand useAuthStore (persisted to localStorage as elmoghazi-auth)

Password Change:

To change the password, edit /api/auth/login/route.ts :

```
const VALID_PASSWORD = '2137' // ← Change this
```

Role Permissions:

Feature	Doctor	Secretary
Dashboard	✓	✗ (auto-redirect to patients)
Patients	✓	✓
Laser	✓	✓
Finance	✓	✗
More section	✓	✗
Settings	✓	✗

Auth Code (`src/app/api/auth/login/route.ts`):

```
import { db } from '@/lib/db'
import { NextResponse } from 'next/server'

const VALID_PASSWORD = '2137'

export async function POST(request: Request) {
  try {
    const body = await request.json()
    const { email, password, role } = body

    if (!password) {
      return NextResponse.json({ error: 'Password is required' }, { status: 400 })
    }

    if (password !== VALID_PASSWORD) {
      return NextResponse.json({ error: 'Invalid password' }, { status: 401 })
    }

    if (email) {
      const user = await db.user.findUnique({ where: { email } })
      if (!user) return NextResponse.json({ error: 'User not found' }, { status: 404 })
      if (!user.active) return NextResponse.json({ error: 'Account is deactivated' }, { status: 403 })
      if (user.password !== password) return NextResponse.json({ error: 'Invalid password' }, { status:
401 })
      const { password: _, ...userWithoutPassword } = user
      return NextResponse.json({ user: userWithoutPassword })
    }

    if (role) {
      const roleEmail = role === 'doctor' ? 'doctor@elmoghazi.com' : 'secretary@elmoghazi.com'
      let user = await db.user.findUnique({ where: { email: roleEmail } })

      if (!user) {
        user = await db.user.create({
          data: {
            name: role === 'doctor' ? 'السكتريرة' : 'دكتور المغازي',
            email: roleEmail,
            password: VALID_PASSWORD,
            role: role,
            active: true,
          },
        })
      }

      if (!user.active) return NextResponse.json({ error: 'Account is deactivated' }, { status: 403 })
      const { password: _, ...userWithoutPassword } = user
      return NextResponse.json({ user: userWithoutPassword })
    }

    return NextResponse.json({ error: 'Email or role is required' }, { status: 400 })
  } catch (error) {
    console.error('Login error:', error)
    return NextResponse.json({ error: 'Internal server error' }, { status: 500 })
  }
}
```

15. Business Logic | المنطق التشغيلي

15.1 Financial Auto-Recording

When a visit/session is created, a financial transaction is automatically created:

Action	Visit Type	Financial Category	Default Price
Add visit from patient profile	checkup	كشف	200 EGP
Add visit from patient profile	revisit	إعادة	80 EGP
Add visit from patient profile	session	جلسات	manual
Add session from patient profile	—	جلسات	manual
Pay laser session	—	ليزر	from session
Pay follow-up visit	—	متابعة	from visit
New patient + visit	checkup	كشف	manual
New patient + visit	revisit	إعادة	manual
New patient + session	—	جلسات	manual

15.2 Financial Category Exclusions

`personal` category transactions are EXCLUDED from clinic revenue calculations

Used for personal expenses/income tracking

15.3 Timezone Handling

Server (Vercel): Runs in UTC

Display: All dates/times shown in Cairo timezone (Africa/Cairo)

Storage: Prisma stores DateTime as UTC

DST: Automatically handled by `Intl.DateTimeFormat` with `timeZone: 'Africa/Cairo'`

15.4 Search Logic

Normalize Arabic: Remove diacritics, normalize alef/yaa/taa

Try exact match: Substring after normalization (highest score)

Try fuzzy match: Character-by-character order match (lower score)

Score & rank: Higher score = better match

15.5 Data Loading

On app load (`useEffect`), all data is fetched in parallel:


```

Promise.allSettled([
  /patients, /visits, /sessions, /services, /notes, /alerts,
  /reminders, /laser/records, /laser/packages, /laser/settings,
  /finance/transactions, /appointments, /waiting, /inventory/items,
  /medications, /prescriptions, /backups, /notifications, /doctors,
  /follow-up/records
])

```

Each response is normalized: `res?.data || res?.patients || res?.visits || ...`

15.6 State Management

Zustand stores: Auth + Clinic settings (persisted to localStorage)

React state: All data (patients, visits, etc.) — NOT persisted, fetched from API on load

Optimistic updates: UI updates immediately, then API call confirms

15.7 Combo Visit Types

`checkout_session` : Creates 1 visit (type=checkout) + 1 session + 2 transactions (كشف + جلسات)

`revisit_session` : Creates 1 visit (type=revisit) + 1 session + 2 transactions (إعادة + جلسات)

15.8 Patient File Sections (Tabs)

Quick info + timeline : **(Overview)** نظرة عامة

Add/edit/delete visits : **(Visits)** الزيارات

Add/edit/delete sessions : **(Sessions)** الجلسات

View laser records : **(Laser)** الليزر

Patient reminders : **(Reminders)** التذكيرات

Patient-specific notes : **(Notes)** الملاحظات

before/after/general photos : **(Photos)** الصور

Follow-up records : **(Follow-up)** المتابعة

15.9 Main Navigation Tabs

Home with stats and quick actions : **(Dashboard)** الرئيسية

Patient list and profiles : **(Patients)** المرضى

Laser records management : **(Laser)** الليزر

Financial transactions and reports : **(Finance)** المالية

16 sub-sections (see Section 3.5) : **(More)** المزيد

15.10 Backup/Restore Order

When restoring, data is processed in dependency order:

Delete (reverse order): Children first, parents last

Create (forward order): Parents first, children last

Skip duplicates: `skipDuplicates: true` in Prisma createMany

Strip virtual fields: Remove `_count` , `patient` , `doctor` , etc. before insert

Contact & Support

GitHub Repo: `sayedhassan589-art/elmoghazi-clinic`

Login Password: `2137`

Database: Neon PostgreSQL

Hosting: Vercel

Critical Reminders for Rebuild

Always use `cairoDateTime()` when sending dates to API — never send bare "YYYY-MM-DD"

Always use `getVisitCategory()` for visit type → financial category mapping — never inline ternary

Always use `normalizePhone()` on phone input fields — for WhatsApp compatibility

Always use `waPhone()` for WhatsApp links — never raw phone numbers

All dates displayed with `timeZone: 'Africa/Cairo'` — never UTC

`addItem()` has `silent` parameter — use `true` when showing custom toast

Personal transactions excluded from clinic revenue — `t.category !== 'personal'`

Login password is `2137` — hardcoded in `/api/auth/login/route.ts`

RTL is mandatory — `dir="rtl"` on `<html>` in layout

Fonts: Cairo + Playfair Display — from Google Fonts in layout.tsx

Document Version: 1.0 **Last Updated:** 2026-06-16 **Application Version:** Latest commit on `main` branch